Data Structures in Python: Lists, Tuples, Dictionaries and Sets

An Assignment REPORT

Assignment 3 – CO3

A Report submitted in partial fulfilment of the requirements for the Course of

CSA0807 Python Programming

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE ENGINEERING

Submitted by

P V GIRIDHAR

Under the Supervision of

Dr. Haripriya Anandakumar

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

NOVEMBER 2026

GITHUB LINK: https://github.com/GIRIDHAR-PV/CSA0807-ASSIGNEMENT-3/tree/main/ASSIGNEMENT%203

Problem Statement – Data Structures for Unstructured Data

Real-world data rarely arrives in neat tables: rainfall readings, bus-stop coordinates, farmer complaints, student registrations, sensor logs, train timetables and phone contacts all need the right Python container. This assignment (CO3) uses lists, tuples, dictionaries and sets to store, access and manipulate such data, and asks for every solution to be designed, implemented, analysed and evaluated (Design → Implementation → Analysis → Evaluation).

Aim of this report

Each of the seven questions below has its own Python file. The table shows which file belongs to which question and what it demonstrates. All outputs and timings in this report were produced by running those files with Python 3.12.3 (standard library only, so requirements.txt is empty).

Q	Python file	What the file demonstrates	Marks
1	Q1.py	Lists and slicing: weekly totals, moving average (naive, running sum, deque)	6
2	Q2.py	Tuples as keys, grid bug, aliasing, shallow and deep copy, memory and timing	7
3	Q3.py	Dictionaries: word frequencies, grouping, dict vs list.count vs Counter	7
4	Q4.py	Sets: clash detection with operators and methods, membership timing	7
5	q5.py	Loop vs list comprehension vs map/filter, generator memory	6
6	Q6.py	Nested timetable, sorted() with lambda, station index	7
7	Q7.py	Contact merging from first principles with dictionary and sets	10
		Total	50

Repository layout: one folder per question (Q1 to Q7) containing the source file, input files, output screenshots and a short note; a README.md gives the name, register number, Python version and the command used to run each program. Each task was committed separately.

Q1 – Lists and Slicing: Daily Rainfall Records (K3 – Apply, 6 marks)

1. Implementation

365 readings are stored in one list (random test data, seed 1). Weekly totals use slicing in steps of 7, the extremes use `max`/`min` with `index`, and the dry days use a comprehension. File: `Q1.py`.

```
def weekly_totals(r):
    return [round(sum(r[i:i+7]), 1) for i in range(0, len(r), 7)]

def ma_naive(r):
    return [sum(r[i:i+7]) / 7 for i in range(len(r) - 6)]

def ma_running(r):
    if len(r) < 7: return []
    s = sum(r[:7]); out = [s / 7]
    for i in range(7, len(r)):
        s += r[i] - r[i-7]
        out.append(s / 7)
    return out
```

Result	Output
Number of weeks	53 (the last week has 1 day)
Wettest week	Week 32, 166.9 mm
Driest week	Week 8, 0 mm
Maximum reading	59.2 mm on day 254
Minimum reading	0 mm on day 1
Dry days	270
Moving-average length	359 (= 365 − 6)

2. Pseudocode – 7-day moving average (running sum)

```
IF length(readings) < 7 THEN RETURN empty list
sum = total of readings[0..6]
result = [sum / 7]
FOR i = 7 TO n-1
    sum = sum + readings[i] - readings[i-7]
    APPEND sum / 7 TO result
RETURN result
```

3. What the slices return

For `readings = [1, 2, ..., 8]`:

- `readings[7:14]` – the 8th to 14th readings, i.e. the second week (days 8–14). Slicing never fails past the end; it simply stops.
- `readings[-7:]` – the last 7 readings, i.e. the final week.
- `readings[::7]` – every 7th reading starting from day 1 (days 1, 8, 15 ...), 53 values for a year.

4. Why a list, and what the operations cost

Readings are ordered by date, stored once and mostly read by position, so a list fits.

Operation	Cost
`append`	O(1) amortised
`pop()` (last item)	O(1)
`insert(i, x)` / `pop(i)`	O(n) – later elements shift
`sort()`	O(n log n)

5. Additions needed for the moving average

There are n − 6 windows. **Naive:** each window needs 6 additions, so 6(n − 6) ≈ 6n. **Running sum:** 6 additions for the first window, then one add and one subtract per step: 6 + 2(n − 7) ≈ 2n.

6. Experiment

n	Naive (s)	Running sum (s)	deque (s)	Additions naive	Additions running
365	0.00009	0.00003	0.00004	2,154	722
36,500	0.00971	0.00274	0.00393	218,964	72,992
365,000	0.11476	0.04316	0.04496	2,189,964	729,992

The operation count drops by about 3×, and the measured time drops by 2.7–3.5×, which matches the derivation. Time grows about 10× when n grows 10×, confirming O(n). **Edge tests:** an empty list returns `[]`; a list of 3 values returns `[]` (no full window); naive and running-sum results are equal for the whole year.

7. Design question – a late reading

`list.insert()` costs O(n) because later elements shift. I measured 100 inserts at the front of a 100,000-item list at 0.00328 s, against 0.000005 s for 100 appends. For 365 items this is negligible, so I would still use a list. If inserts were frequent I would store the readings in a dictionary keyed by date.

8. Alternative concept and complexity

- `deque(maxlen=7)`**:** appending a new reading drops the oldest automatically, so the window sum is updated in O(1) per day. Time O(n); the window needs O(1) space (plus the output). Its measured speed was about the same as the running sum.
- **Dictionary keyed by date:** `{date: mm}` gives O(1) lookup and easy late inserts, but uses more memory (O(n) with hash overhead) and has no slicing by position.
- **Slicing a list:** O(7n) time and O(n) space; the simplest to read.

OUTPUT (python Q1.py):

```
weeks: 53 first 5: [114.6, 97.5, 101.4, 122.7, 75.5]
wettest week 32 166.9 | driest week 8 0
max 59.2 at day 254 | min 0 at day 1
dry days: 270 [1, 4, 5, 7, 8, 9, 10, 11, 12, 15]
MA len 359 first 3 [16.37, 16.37, 14.19]
slices: [2, 3] [6, 7, 8] [1, 4, 7]
equal: True check
empty: [] [] short: [] []
365 naive 0.00009 running 0.00003 deque 0.00005 adds naive 2154 running 722
36500 naive 0.01035 running 0.00288 deque 0.00400 adds naive 218964 running 72992
365000 naive 0.09739 running 0.03198 deque 0.04358 adds naive 2189964 running 729992
insert at front x100 on 100k: 0.00340
append x100: 0.000005
```

Q2 – Tuples, Mutability and Copying: City Bus-Stop Map (K4 – Analyse, 7 marks)

1. Complete the Code – TODO 1

```
good = [[0 for j in range(cols)] for i in range(rows)]
```

The output matches the expected output:

```
bad : [[0, 7, 0, 0], [0, 7, 0, 0], [0, 7, 0, 0]]
good: [[0, 7, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
True False
T. Nagar
```

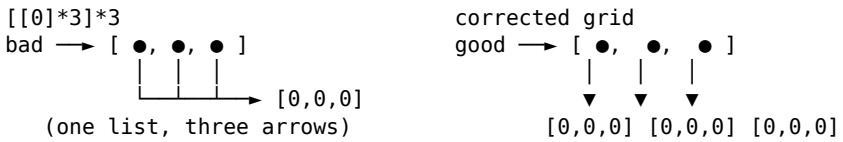
List as a key: `{[13.0418, 80.2341]: "T Nagar"}` raises `TypeError: unhashable type: 'list'`. A dictionary needs a hash that never changes. A list is mutable, so its hash could change after insertion and the entry could never be found again. A tuple is immutable, so its hash is stable.

2. The bug and its correction with is and id()

`[[0]*3]*3` repeats a reference to one inner list three times.

```
bad[0] is bad[1]   -> True   (id(bad[0]) == id(bad[1]))
good[0] is good[1] -> False  (different ids)
```

Memory diagram



3. Aliasing, shallow copy and deep copy

Demonstration	Result
`b = a; b[0] = 100`	Both show `[100, 20, 30]`; `a is b` is True
`copy.copy(original)`, then change `shallow[0][0]`	The original also becomes `[[99, 2], [3, 4]]`
`original[:]` slice copy	Same behaviour: `s[0] is original[0]` is True
`copy.deepcopy(original)`	The original stays `[[1, 2], [3, 4]]`

Shallow vs deep: a shallow copy is enough when the list holds immutable items (numbers, strings, tuples). A deep copy is needed when it holds mutable items (nested lists, dictionaries) that will be changed independently.

Function side effect: Python passes a reference to the object. `change_list(numbers)` appends to the same list the caller holds, so the caller sees `[10, 20, 30, 500]`.

4. Tuple vs list measurements (1,000,000 values)

Measure	List	Tuple
`sys.getsizeof`	8,000,056 bytes	8,000,040 bytes
Creation time	0.0242 s	0.0283 s
100,000 index operations	0.00500 s	0.00505 s

The memory gap is only 16 bytes, so for large data the saving is tiny and the speed is effectively the same. The real benefit of a tuple is safety, not speed.

A list inside a tuple: `my_tuple = ([10, 20], [30, 40]); my_tuple[0].append(50)` works and gives `([10, 20, 50], [30, 40])`. The tuple still holds the same two list objects; only the content of one of them changed. Immutability is shallow.

5. Conclusion

Use a tuple for fixed records such as a (latitude, longitude) pair or a dictionary key. Immutability protects against accidental edits, bugs from shared references and unhashable-key errors.

6. Alternative concept – dictionary grid

Store `{(row, col): count}` for non-empty zones only. Access is O(1) average in both designs. Space is O(rows × cols) for the list of lists but only O(number of non-empty zones) for the dictionary, which wins when most zones are empty. It also avoids the shared-row bug.

OUTPUT (python Q2.py):

```
bad: [[0, 7, 0, 0], [0, 7, 0, 0], [0, 7, 0, 0]]
good: [[0, 7, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
True False
T. Nagar
Error: unhashable type: 'list'

Aliasing
a: [100, 20, 30]
b: [100, 20, 30]
Same object: True

Shallow copy
Original: [[99, 2], [3, 4]]
Shallow: [[99, 2], [3, 4]]

Deep copy
Original: [[1, 2], [3, 4]]
Deep: [[100, 2], [3, 4]]

Function result: [10, 20, 30, 500]
Latitude: 13.0827 Longitude: 80.2707
Latitude: 13.0418 Longitude: 80.2341

Memory comparison
List memory: 8000056
Tuple memory: 8000040

Creation time
List: 0.027309950000017125
Tuple: 0.02764594100000295
List indexing: 0.005112058999998226
Tuple indexing: 0.005044862000005423

Tuple containing lists: ([10, 20, 50], [30, 40])
```

Q3 – Dictionaries: Farmer Helpline Complaint Analysis (K4 – Analyse, 7 marks)

1. Complete the Code – TODO 1 and TODO 2

```
def frequencies(texts, stop):
    counts = {}
    for sentence in texts:
        for word in sentence.lower().split():
            word = word.strip(string.punctuation)
            if word != "" and word not in stop:
                counts[word] = counts.get(word, 0) + 1
    return counts

def top(counts, k):
    result = sorted(counts.items(), key=lambda item: (-item[1], item[0]))
    return result[:k]
```

Output: `[('seeds', 3), ('price', 2), ('received', 2)]` – matches the expected output.

Why price comes before received: both occur twice. The sort key is `(-count, word)`, so the counts tie and the words are compared alphabetically; `p` comes before `r`.

Grouping by district with `setdefault`:

```
Chennai : ['Seeds not received', 'Water shortage']
Madurai : ['Fertiliser price high', 'Seeds delayed']
Salem   : ['Crop damage']
```

2. Design and analysis

- **How a dictionary finds a key:** Python computes `hash(key)`, uses it to pick a slot in a hash table and compares the key there. If the slot holds a different key it probes the next slot. The table is resized to stay sparse, so the average number of probes stays constant and lookup is O(1) whatever the size.
- **Dictionary vs `list.count()`:** a dictionary pass touches each word once, O(n). `list.count()` for each of d distinct words scans the whole list every time, O(n·d).
- **Key order:** from Python 3.7 a dictionary remembers insertion order (a language guarantee). Updating an existing key does not change its position.
- **Views:** `keys()`, `values()` and `items()` return live views of the dictionary, not copies. They reflect later changes, and `items()` yields (key, value) tuples.

3. Experiment

Complaints	Dictionary (s)	list.count (s)	Counter (s)
1,000	0.00036	0.00051	0.00014
10,000	0.00366	0.00514	0.00149
100,000	0.04123	0.05689	0.01500

`Counter` is about 2.7× faster than the plain dictionary loop because its counting loop is written in C. `list.count` looks close here only because the templates contain just 11 distinct words (d is small); with thousands of distinct words it would be far slower, as O(n·d) predicts.

Edge tests: `frequencies([""], STOP)` returns `{}`; `"SEEDS, seeds! Seeds."` returns `{'seeds': 3}`, because text is lower-cased and punctuation is stripped.

4. Engineering decision

For 100,000 complaints a day I would deploy `collections.Counter`: it is the fastest, the shortest to write and still O(n). I would keep a hand-written dictionary loop only when extra per-word logic is needed.

5. Alternative concept

`list.count()` for every distinct word is O(n·d) time and O(d) extra space. `Counter` is O(n) time and O(d) space, the same as the dictionary loop.

OUTPUT (python Q3.py):

```
Word frequencies: {'seeds': 3, 'received': 2, 'delayed': 1, 'fertiliser': 1, 'price': 2, ...
Top words: [('seeds', 3), ('price', 2), ('received', 2)]

District complaints:
Chennai : ['Seeds not received', 'Water shortage']
Madurai : ['Fertiliser price high', 'Seeds delayed']
Salem : ['Crop damage']

Complaints: 1000
Dictionary time: 0.0008134109999957673
List count time: 0.0008664169999974547
Counter time: 0.00024380100001053506

Complaints: 10000
Dictionary time: 0.005570210000001907
List count time: 0.006491659999994681
Counter time: 0.002669711000010011

Complaints: 100000
Dictionary time: 0.04761242899999729
List count time: 0.05106661599998574
Counter time: 0.013594405999981518

Empty text: {}
Case and punctuation: {'seeds': 3}
```

Q4 – Sets: Elective Registration Clash Detection (K4 – Analyse, 7 marks)

1. Complete the Code – TODO 1 (set operators)

```
both        = ai_elective & iot_elective
exactly_one = ai_elective ^ iot_elective
neither     = all_students - (ai_elective | iot_elective)

clash      : ['192311004', '192311009']
exactly one: ['192311001', '192311002', '192311007', '192311010']
neither    : ['192311003', '192311005', '192311006', '192311008']
```

This matches the expected output. The **set-method version** (`intersection`, `symmetric_difference`, `difference` with `union`) gave identical results. **Duplicates:** `set(["...001", "...001", "...004", "...009", "...004"])` keeps three unique numbers.

2. Venn diagrams

```
A = AI only: 001, 007     B = IoT only: 002, 010     A ∩ B: 004, 009

Intersection  A ∩ B        -> only the overlap {004, 009}
Symmetric difference A ^ B -> both circles minus the overlap {001, 007, 002, 010}
Neither  U − (A ∪ B)       -> outside both circles {003, 005, 006, 008}
```

3. Design and analysis

- **Why duplicates vanish:** a set is a hash table of unique keys; adding an item that is already there changes nothing.
- **Why `x in s` is faster than `x in list`:** a set hashes x and jumps to its slot, O(1) average. A list compares x with each item in turn, O(n).
- **Lists vs tuples inside sets:** set elements must be hashable. Lists are mutable and unhashable, so they cannot be elements; tuples of hashable items can.

4. Experiment (100 lookups of the last item)

Size	x in list (s)	x in set (s)
10,000	0.00651	0.0000054
100,000	0.07432	0.0000070
1,000,000	0.66680	0.0000058

List time grows linearly; set time stays flat. **Common students (1,000 vs 1,000 items):** nested loops took 0.04599 s and set `&` took 0.0000595 s, about 770× faster. **Edge tests:** `set() & set()` is `set()`; an elective intersected with itself returns itself. **Manual check:** A = {1,2,3}, B = {2,3,4}: both = {2,3}, exactly one = {1,4}, neither (universe 1–5) = {5}; the program printed exactly the same.

5. Conclusion

Convert a list to a set before processing when you need fast membership tests, uniqueness or set algebra. You lose the order, the duplicate counts and the positions.

6. Alternative concept

Nested loops on lists take O(n·m) time. Sorting both lists and walking through them together takes O(n log n + m log m). Sets take O(n + m) on average. Extra space is O(1) for nested loops, O(1) to O(n) for the sorted walk (depending on in-place sorting) and O(n + m) for sets.

OUTPUT (python Q4.py):

```
clash      : ['192311004', '192311009']
exactly one: ['192311001', '192311002', '192311007', '192311010']
neither    : ['192311003', '192311005', '192311006', '192311008']

Using set methods
clash: ['192311004', '192311009']
exactly one: ['192311001', '192311002', '192311007', '192311010']
neither: ['192311003', '192311005', '192311006', '192311008']

Unique registrations: ['192311001', '192311004', '192311009']

Size: 10000
List membership: 0.006427019999989625
Set membership: 5.52800000264142e-06

Size: 100000
List membership: 0.06508655599998292
Set membership: 6.940999980997731e-06

Size: 1000000
List membership: 0.6633760119999863
Set membership: 6.130999992137731e-06

Nested loop time: 0.03641274999998245
Set intersection time: 4.6510999993643054e-05

Empty electives: set()
Identical electives: {'192311001', '192311009', '192311007', '192311004'}

Manual verification
Both: {2, 3}
Exactly one: {1, 4}
Neither: {5}
```

Q5 – List Comprehension: Cleaning Sensor Readings (K3 – Apply, 6 marks)

1. Implementation

Three versions clean the readings outside −50 to 50 °C and convert them to Fahrenheit (F = C × 9/5 + 32): a `for` loop with `append`, list comprehensions, and `filter`/`map` with `lambda`. The alarm minutes use `enumerate` and flag valid readings outside 2–8 °C (minute = index + 1). **Verification:** `loop == comprehension` and `comprehension == map/filter` are both True.

General form: `[expression for item in iterable if condition]`

```
{i + 1: v for i, v in enumerate(readings) if -50 <= v <= 50}   # dictionary
{v for v in readings if -50 <= v <= 50}                        # set
```

When a comprehension is harder to read than a loop: when it has nested loops, several conditions or a long expression. A plain loop with named steps is clearer then.

List comprehension vs generator expression: square brackets build the whole list in memory at once; round brackets build a lazy iterator that produces one value at a time and can be used only once.

2. Timing

Readings	Loop (s)	Comprehension (s)	map/filter (s)
10,000	0.00257	0.00295	0.00312
100,000	0.02405	0.02912	0.02540
1,000,000	0.16598	0.20037	0.19829

3. Memory and edge tests

For 100,000 values a list comprehension used 800,984 bytes and a generator expression 192 bytes. All-valid `[3, 4, 5, 6, 7]` returned all five readings in Celsius and Fahrenheit; all-invalid `[60, 70, -60, -80]` returned two empty lists.

4. Conclusion

A list comprehension is not always the best choice. In my measurements the plain loop was fastest, because my comprehension version scans the data twice (once for Celsius, once for Fahrenheit) while the loop makes one pass. A single-pass version would close the gap. Comprehensions are best when they stay short and readable; generators are best when the values are consumed only once.

5. Alternative concept

`sum(c for c in readings if -50 <= c <= 50)` is O(n) time like the list version, but needs O(1) memory instead of O(n): 192 bytes against about 800 KB in my measurement.

OUTPUT (python q5.py):

```
Loop result: [-36.276537218160655, 43.034497055641154, -10.27025088531586, -47.5350806845...
Comprehension result: [-36.276537218160655, 43.034497055641154, -10.27025088531586, -47.5...
Map result: [-36.276537218160655, 43.034497055641154, -10.27025088531586, -47.53508068454...

Verification
True
True

Alarm minutes: [2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 13, 14, 15, 17, 18, 19, 21, 22, 24, 25, 2...
Temperature dictionary: [(2, -36.276537218160655), (3, 43.034497055641154), (4, -10.27025...
Unique valid temperatures: 8327

Size: 10000
Loop time: 0.002443773999999621
Comprehension time: 0.0028907650000178364
Map filter time: 0.0030092850000187354

Size: 100000
Loop time: 0.025439407000021674
Comprehension time: 0.02917092800001342
Map filter time: 0.021397041999989597

Size: 1000000
Loop time: 0.14988001300000064
Comprehension time: 0.199237882999995
Map filter time: 0.192466048

Memory comparison
List memory: 800984
Generator memory: 192

All valid: ([3, 4, 5, 6, 7], [37.4, 39.2, 41.0, 42.8, 44.6])
All invalid: ([], [])
```

Q6 – Nested Data: A Railway Timetable (K4 – Analyse, 7 marks)

1. Structure

```
timetable = { train_no: { "name": str,
                          "stops": [ (station, arrival, departure), ... ],
                          "days": { "Mon", "Tue", ... } } }
```

Third station departure of train 12001: `timetable["12001"]["stops"][2][2]` gives `"11:10"`.

Why each type:

- Dictionary – look up a train by number in O(1) and name its fields by label.
- List – the stops are ordered.
- Tuple – a stop is a fixed record that should not be edited.
- Set – running days are unique, unordered and tested with `in`.

2. Results (5 trains)

Trains at Chennai Central on Monday, sorted by departure with `sorted(..., key=lambda x: x[3])`:

Train	Name	Arrival	Departure
12001	Chennai Express	06:00	06:05
12002	Coimbatore Express	07:00	07:10
12004	Mysore Express	09:00	09:10
12005	Kerala Express	10:00	10:05

Train 12003 does not run on Monday.

3. Work for one station query

A query checks every train and, for trains running that day, scans its stops: O(T × s) for T trains with s stops each, i.e. O(total stops).

4. Index design

`index[station] = [(train_no, name, departure, days), ...]`. A query goes straight to the list for that station and filters by day, so the work is O(trains at that station).

5. Experiment (random timetables, 5 stops per train, 50 stations, 200 queries each)

Trains	Scan per query	Indexed per query	Speed-up	Index memory
100	0.014 ms	below 0.001 ms	about 28×	9,536 B
1,000	0.141 ms	0.005 ms	about 29×	47,648 B
10,000	1.703 ms	0.051 ms	about 34×	436,192 B

On the 5-train sample, 1,000 queries took 0.00134 s without the index and 0.00034 s with it. Index memory counts the containers only, not the tuples inside. **Edge tests:** an unknown station returns `[]`; Madurai on Monday returns `[]` because train 12003 does not run on Mondays.

6. Conclusion

The index does the searching once, in advance, and groups trains by station, so a query reads only the relevant entries. The trade-off is extra memory and the need to update the index when the timetable changes.

7. Alternative concept

A flat list of `(train, station, arrival, departure, days)` tuples scanned for every query costs O(total stops) time per query and O(total stops) space, with the train name and days repeated in every row. The nested design with an index costs O(trains at the station) per query, plus O(total stops) extra space for the index.

OUTPUT (python Q6.py):

```
RAILWAY TIMETABLE

Train: 12001
Name: Chennai Express
Running days: {'Thu', 'Tue', 'Fri', 'Wed', 'Mon'}
Chennai Central 06:00 06:05
Katpadi 08:00 08:05
Bangalore 11:00 11:10

Train: 12002
Name: Coimbatore Express
Running days: {'Fri', 'Wed', 'Mon'}
Chennai Central 07:00 07:10
Salem 10:00 10:05
Coimbatore 12:00 12:10

Train: 12003
Name: Madurai Express
Running days: {'Thu', 'Tue', 'Sat'}
Chennai Central 08:00 08:05
Trichy 12:00 12:05
Madurai 14:00 14:10

Train: 12004
Name: Mysore Express
Running days: {'Thu', 'Tue', 'Sat', 'Fri', 'Wed', 'Mon'}
Chennai Central 09:00 09:10
Katpadi 11:00 11:05
Mysore 15:00 15:10

Train: 12005
Name: Kerala Express
Running days: {'Wed', 'Sun', 'Mon'}
Chennai Central 10:00 10:05
Salem 13:00 13:05
Ernakulam 17:00 17:10

Trains at Chennai Central on Mon
('12001', 'Chennai Express', '06:00', '06:05')
('12002', 'Coimbatore Express', '07:00', '07:10')
... (output shortened; full output in the GitHub repository)
```

Q7 – Contact-Merging Program from First Principles (K5 – Evaluate, 10 marks)

Stage 1 – Interpret the problem (2 marks)

- **Input:** contacts from two phones and a SIM, each with a name, a list of phone numbers and a list of e-mails. Any field may be missing.
- **Output:** one record per person with all their names, numbers and e-mails.
- **Same-person rule:** two contacts are the same person if they share at least one normalised phone number. Names are not used to decide, because spellings vary and different people can share a name.
- **Data structures:** a dictionary (key = normalised number) of records; sets for names, numbers and e-mails (no duplicates); a phone index dictionary (number → record key).
- **Measurable requirements:** (1) `+91 98400 12345` and `9840012345` give the same key; (2) every input number and e-mail appears exactly once in the output; (3) a contact with no number is kept, not dropped; (4) 100,000 contacts are merged in about a second or less.

Stage 2 – Design (3 marks)

Normalise a phone number: remove every non-digit; if 12 digits starting with 91, drop the 91; if 11 digits starting with 0, drop the 0. **Normalise a name:** lower-case, trim, remove a leading Dr/Mr/Mrs/Ms, keep letters and spaces.

```
FOR each contact
    names  = {name} if name present
    phones = {normalise(p) for p in contact phones if not empty}
    emails = {lowercase(e) for e in contact emails if not empty}
    matched = {phone_index[p] for p in phones if p in phone_index}
    IF matched is empty
        key = smallest phone, or "missing_i" if there is no phone
        merged[key] = empty record
    ELSE
        key = smallest of matched
        merge every other matched record into merged[key] and delete it
    add names, phones, emails to merged[key]
    FOR each phone in merged[key]: phone_index[phone] = key
RETURN merged
```

Stage 3 – Implement and measure (3 marks)

Hand-verified small test (6 contacts):

Merged ID	Names	Numbers	E-mails
9840012345	Dr. Ravi, Ravi, Ravi Kumar	9000011111, 9840012345	ravi@gmail.com, ravi@yahoo.com
9876543210	Arun	9876543210	arun@gmail.com
9876543211	Priya	9876543211	–
missing_5	Unknown	–	unknown@gmail.com

Six contacts become four people, which I confirmed by hand: the three Ravi entries merged through the shared number 9840012345, and the extra number 9000011111 joined them. **Missing-field test:** a

contact with no phone becomes its own `missing_` record, and a contact with an empty name is kept with an empty name set.

Input contacts	Merged contacts	Time
1,000	1,000	0.0043 s
100,000	50,000	0.4607 s

The 100,000 test deliberately reuses 50,000 numbers, so exactly half merge. Time grew about 106× for 100× more data, which is roughly linear.

Stage 4 – Defend the design (2 marks)

- **Dictionary vs comparing every pair:** pair comparison needs $n(n - 1)/2$ checks, $O(n^2)$. The dictionary does one average O(1) lookup per number, so the whole job is O(n).
- **Two different people with the same name:** names are not the merge key, only phone numbers are. Two people called Ravi with different numbers stay as two records.
- **Remaining limitation:** a person with no phone number, or with different numbers on different phones and no shared number, is never merged. A shared number (for example a family landline) would wrongly merge two people. E-mail could be added as a second matching key.

Alternative concept – pairwise comparison

Compare every pair of contacts and union those sharing a number. Both versions were run on the same data:

n	Pairwise (s)	Dictionary (s)
1,000	0.0428	0.0030
5,000	1.1199	0.0190

Going from 1,000 to 5,000 (5× more data) made the pairwise version about 26× slower, matching $O(n^2)$. Extrapolating, 100,000 contacts would take roughly 450 s pairwise, against 0.46 s for the dictionary (the 100,000 pairwise figure is an estimate, not a direct run). Space is O(n) for both.

Overall Conclusion

Working through the seven questions showed that the choice of container decides both correctness and speed. Lists suit ordered readings but cost O(n) for inserts; tuples protect fixed records and can be dictionary keys; dictionaries and sets give O(1) average lookup and turned $O(n \cdot d)$, $O(n \cdot m)$ and $O(n^2)$ searches into O(n) ones, as the measurements confirmed (up to about 770× faster for set intersection and about 34× for the station index). Comprehensions and generators are tools to be chosen by readability and memory, not automatically, and the shared-reference bug in [[0]*cols]*rows is a reminder to check aliasing whenever mutable objects are copied. The contact-merging design showed how a dictionary keyed by a normalised value replaces pairwise comparison, with its limitations stated openly.